



BookSearch MicroService

- Application Name : **MyBookSearchMS**
- Ports : 8000 , 8100 ,8200

Steps to Develop MyBookSearchMS:

A)Setup the Database:

1) Create and open the database

```
drop database jlcbooksdb;  
create database jlcbooksdb;  
use jlcbooksdb;
```

2) Create the tables required

```
drop table mybooks;
```

```
create table mybooks(  
book_id int primary key,  
book_name char(30),  
author char(15),  
publications char(15),  
category char(15)  
);
```

```
drop table mybookratings;
```

```
create table mybookratings(  
book_id int primary key,  
avg_rating double,  
number_of_searches int  
);
```

```
drop table mybookinventory;
```

```
create table mybookinventory(  
book_id int primary key,  
books_available int  
);
```



3) Insert some sample records

```
insert into mybooks values(101,'Master Spring Boot','Srinivas','JLC','Spring');
insert into mybooks values(102,'Master MicroServices','Sri','JLC','Spring');
insert into mybooks values(103,'Master Angular','Srinivas','JLC','Web');
insert into mybooks values(104,'Master React','Sri','JLC','Web');
insert into mybooks values(105,'Master Spring Framework','Srinivas','JLC','Spring');
insert into mybooks values(106,'Master Spring Security','Vas','JLC','Spring');
insert into mybooks values(107,'Master Vue JS','Srinivas','JLC','Web');
insert into mybooks values(108,'Node JS','Vas','JLC','Web');
insert into mybooks values(109,'Master Spring Rest','Srinivas','JLC','Spring');
```

```
insert into mybookratings values(101,3.5,99);
insert into mybookratings values(102,3.5,99);
insert into mybookratings values(103,3.5,99);
insert into mybookratings values(104,3.5,99);
insert into mybookratings values(105,3.5,99);
insert into mybookratings values(106,3.5,99);
insert into mybookratings values(107,3.5,99);
insert into mybookratings values(108,3.5,99);
insert into mybookratings values(109,3.5,99);
```

```
insert into mybookinventory values(101,50);
insert into mybookinventory values(102,50);
insert into mybookinventory values(103,50);
insert into mybookinventory values(104,50);
insert into mybookinventory values(105,50);
insert into mybookinventory values(106,50);
insert into mybookinventory values(107,50);
insert into mybookinventory values(108,50);
insert into mybookinventory values(109,50);
```

4) Select and see the sample records

```
select * from mybooks
select * from mybookratings;
select * from mybookinventory;
```

B)Setup the Boot Project

5) Copy BookPriceMS and Paste as BookSearchMS

6) Remove All except the following

- a) pom.xml
- b) application.properties
- c) JLCBookPriceConfig.java
- d) BookPriceApplication.java

7) Update pom.xml

```
*** Change Names
...
<groupId>com.jlcindia.booksearch</groupId>
<artifactId>BookSearchMS</artifactId>
....
<name>MyBookSearchMS</name>
....
<finalName>MyBookSearchMS</finalName>
```

8) Update application.priperities

```
spring.application.name=MyBookSearchMS
server.port=8000
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbooksdb
```

9) Rename the Package to com.jlcindia.booksearch

10) Rename Spring Bean Configuration Class

```
*** JLCBookSearchConfig.java
```

11) Rename BootApplication Starting Class.

```
*** BookSearchApplication.java
```

C)Develop the BookSearch MicroService Code:

12) Write the JPA Entities

```
*** Book.java
*** BookRating.java
*** BookInventory.java
```



13) Write DAO Layer components

```
*** BookDAO.java  
*** BookRatingDAO.java  
*** BookInventoryDAO.java
```

14) Write DTO

```
*** BookPriceInfo.java  
*** BookInfo.java
```

15) Write Service Layer components

```
*** BookService.java  
*** BookServiceImpl.java
```

16) Write Controller Layer component

```
*** BookController.java
```

17) Start the Boot Application

18) Open the Swagger UI :

<http://localhost:8000/swagger-ui.html>

GET : <http://localhost:8000/mybooks/Srinivas/Spring>

GET : <http://localhost:8000/mybook/101>

PUT : <http://localhost:8000/updateBookRating>

PUT : <http://localhost:8000/updateBookInventory>

19) Test with Browser

<http://localhost:8000/mybooks/Srinivas/Web>

<http://localhost:8000/mybooks/Srinivas/Spring>

<http://localhost:8000/mybook/101>

<http://localhost:8000/mybook/102>



BookSearchMS: Files required

1. BookController.java	2. BookPriceInfo.java
3. BookInfo.java	4. BookService.java
5. BookServiceImpl.java	6. BookDAO.java
7. BookRatingDAO.java	8. BookInventoryDAO.java
9. Book.java	10. BookRating.java
11. BookInventory.java	12. JLCBookSearchConfig.java
13. StartMyBookSearchMS.java	14. application.properties
15. pom.xml	

1. BookController.java

```
package com.jlcindia.booksearch;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import io.swagger.annotations.ApiOperation;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
@CrossOrigin
public class BookController {

    static Logger log = LoggerFactory.getLogger(BookController.class);

    @Autowired
    BookService bookService;

    @GetMapping("/mybooks/{author}/{category}")
    @ApiOperation(value = "getBooks", response = List.class, notes = "Returns List of Books for given Author and Category")
    public List<Book> getBooks(@PathVariable String author, @PathVariable String category) {
        log.info("---BookController---getBooks()-----");
        System.out.println(author + "\t" + category);

        return bookService.getBooks(author, category);
    }
}
```



```
@GetMapping("/mybook/{bookId}")
@ApiOperation(value = " getBookById", response = BookInfo.class, notes = "Returns
BookInfo for given BID")
public BookInfo getBookById(@PathVariable Integer bookId) {
log.info("---BookController---getBookById()-----");

return bookService.getBookInfo(bookId);
}

@PutMapping("/updateBookRating")
@ApiOperation(value = " updateBookRating", response = void.class, notes =
"updateBookRating")
public void updateBookRating(@RequestBody BookRating bookRating) {
System.out.println("-----BookController-----updateBookRating()-----");

bookService.updateBookRating(bookRating);
}

@PutMapping("/updateBookInventory")
@ApiOperation(value = " updateBookInventory", response = void.class, notes =
"updateBookInventory")
public void updateBookInventory(@RequestBody BookInventory bookInventory) {
System.out.println("-----BookController-----updateBookInventory()-----");

bookService.updateBookInventory(bookInventory);
}
}
```

2.BookPriceInfo.java

```
package com.jlcindia.booksearch;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class BookPriceInfo {
    private Integer bookId;
    private double price;
    private double offer;

    //Constrcutors
    //Setetrs and Getters
    //toString()
}
```



3.BookInfo.java

```
package com.jlcindia.booksearch;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class BookInfo {
    private Integer bookId;
    private String bookName;
    private String author;
    private String publications;
    private String category;

    private double price;
    private double offer;

    private double avgRating;
    private int numberOfSearches;

    private int booksAvailable;

    //Constrcutors
    //Setetrs and Getters
    //toString()
}
```

4.BookService.java

```
package com.jlcindia.booksearch;

import java.util.List;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public interface BookService {
    public List<Book> getBooks(String author,String category);
    public BookInfo getBookInfo(Integer bookId);
    public void updateBookRating(BookRating bookRating);
    public void updateBookInventory(BookInventory bookInventory);
}
```



5. BookServiceImpl.java

```
package com.jlcindia.booksearch;

import java.util.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Service
@Transactional
public class BookServiceImpl implements BookService{

    static Logger log=LoggerFactory.getLogger(BookServiceImpl.class);

    @Autowired
    BookInventoryDAO bookInventoryDAO;

    @Autowired
    BookRatingDAO bookRatingDAO;

    @Autowired
    BookDAO bookDAO;

    @Override
    public List<Book> getBooks(String author, String category) {
        List<Book> mybooks=new ArrayList<>();

        if(author.equals("All Authors") && category.equals("All Categories")) {
            mybooks = bookDAO.findAll();
        }else if(author.equals("All Authors") && ! category.equals("All Categories")) {
            mybooks = bookDAO.getBooksByCategory(category);
        } else if(! author.equals("All Authors") && category.equals("All Categories")) {
            mybooks = bookDAO.getBooksByAuthor(author);
        }else {
            mybooks = bookDAO.getBooksByAuthorAndCategory(author, category);
        }
        return mybooks;
    }
}
```



```
public BookInfo getBookInfo(Integer bookId) {

BookInfo bookInfo=new BookInfo();

    //1. Book Details
    Book book=bookDAO.findById(bookId).get();
    bookInfo.setBookId(book.getBookId()); //1
    bookInfo.setBookName(book.getBookName()); //2
    bookInfo.setAuthor(book.getAuthor()); //3
    bookInfo.setPublications(book.getPublications()); //4
    bookInfo.setCategory(book.getCategory()); //5

    //2. Book Rating Details
    BookRating bookRating=bookRatingDAO.findById(bookId).get();

    bookInfo.setAvgRating(bookRating.getAvgRating()); //6
    bookInfo.setNumberOfSearches(bookRating.getNumberOfSearches()); //7

    //3. Book Inventory Details
    BookInventory bookInventory=bookInventoryDAO.findById(bookId).get();
    bookInfo.setBooksAvailable(bookInventory.getBooksAvailable()); //8

    //4. Book Price Details - Invoking BookPriceMS
    RestTemplate bookPriceRest=new RestTemplate();
    String endpoint="http://localhost:9000/bookPrice/"+bookId;
    BookPriceInfo bpInfo=bookPriceRest.getForObject(endpoint, BookPriceInfo.class);
    bookInfo.setPrice(bpInfo.getPrice()); //9
    bookInfo.setOffer(bpInfo.getOffer()); //10

    return bookInfo;
}

@Override
public void updateBookRating(BookRating bookRating) {
    bookRatingDAO.save(bookRating);
}

@Override
public void updateBookInventory(BookInventory bookInventory) {
    bookInventoryDAO.save(bookInventory);
}
}
```



6.BookDAO.java

```
package com.jlcindia.booksearch;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Repository
public interface BookDAO extends JpaRepository<Book,Integer> {
    public List<Book> getBooksByAuthorAndCategory(String author,String category);
    public List<Book> getBooksByAuthor(String author);
    public List<Book> getBooksByCategory(String category);
}
```

7.BookRatingDAO.java

```
package com.jlcindia.booksearch;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Repository
public interface BookRatingDAO extends JpaRepository<BookRating,Integer> {
}
```

8.BookInventoryDAO.java

```
package com.jlcindia.booksearch;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Repository
public interface BookInventoryDAO extends JpaRepository<BookInventory,Integer> {
}
```



9.Book.java

```
package com.jlcindia.booksearch;

import javax.persistence.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Entity
@Table(name="mybooks",schema = "jlcbooksdb")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name="book_id")
    private Integer bookId;

    @Column(name="book_name")
    private String bookName;

    @Column(name="author")
    private String author;

    @Column(name="publications")
    private String publications;

    @Column(name="category")
    private String category;

    //Constrcutors
    //Setetrs and Getters
    //toString()
}
```

10.BookRating.java

```
package com.jlcindia.booksearch;

import javax.persistence.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



```
@Entity
@Table(name="mybookratings",schema = "jlcbooksdb")
public class BookRating {

    @Id
    @Column(name="book_id")
    private Integer bookId;

    @Column(name="avg_rating")
    private double avgRating;

    @Column(name="number_of_searches")
    private int numberOfSearches;

    //Constrcutors
    //Setetrs and Getters
}
```

11.BookInventory.java

```
package com.jlcindia.booksearch;

import javax.persistence.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Entity
@Table(name="mybookinventory",schema = "jlcbooksdb")
public class BookInventory {
    @Id
    @Column(name="book_id")
    private Integer bookId;

    @Column(name="books_available")
    private int booksAvailable;

    //Constrcutors
    //Setetrs and Getters
}
```



12.JLCBookSearchConfig.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import com.google.common.base.Predicates;

import springfox.documentation.builders.PathSelectors;
import springfox.documentation.builders.RequestHandlerSelectors;
import springfox.documentation.service.ApiInfo;
import springfox.documentation.service.Contact;
import springfox.documentation.spi.DocumentationType;
import springfox.documentation.spring.web.plugins.Docket;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCBookSearchConfig implements WebMvcConfigurer {

    private ApiInfo getApiDetails() {
        return new ApiInfo("JLC BookStore- API",
            "BookSearchMS- API - part of BookStore", "1.0", "Free to use ",
            new Contact("Srinivas Dande", "https://www.jlcindia.com", "sri@jlcindia.com"),
            "API Under Free Licence", "https://www.jlcindia.com");
    }
    @Bean
    public Docket api() {
        return new Docket(DocumentationType.SWAGGER_2).select().paths(PathSelectors.any())
            .apis(Predicates.not(RequestHandlerSelectors.basePackage("org.springframework.boot")))
            .build().apiInfo(getApiDetails());
    }
    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("swagger-ui.html").addResourceLocations("classpath:/META-INF/resources/");
        registry.addResourceHandler("/webjars/**").addResourceLocations("classpath:/META-INF/resources/webjars/");
    }
}
```



13. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import springfox.documentation.swagger2.annotations.EnableSwagger2;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableSwagger2
public class BookSearchApplication {
    static Logger log = LoggerFactory.getLogger(BookSearchApplication.class);

    public static void main(String args[]) {
        log.info(" BookSearchMS - Begin ");
        SpringApplication.run(BookSearchApplication.class, args);
        log.info(" BookSearchMS - End ");
    }
}
```

14. application.properties

```
spring.application.name=MyBookSearchMS

server.port=8000

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbooksdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```



15. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
  <groupId>com.jlcindia.booksearch</groupId>
  <artifactId>BookSearchMS</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>
  <name>MyBookSearchMS</name>
  <url>https://www.jlcindia.com</url>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.5.3</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>5.1.6</version>
    </dependency>

    <dependency>
      <groupId>io.springfox</groupId>
      <artifactId>springfox-swagger2</artifactId>
      <version>2.4.0</version>
    </dependency>

    <dependency>
      <groupId>io.springfox</groupId>
      <artifactId>springfox-swagger-ui</artifactId>
      <version>2.4.0</version>
    </dependency>
  </dependencies>
</project>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<build>
<finalName>MyBookSearchMS</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

MicroService Name	Rest Endpoints
MyBookSearchMS	http://localhost:8000/mybooks/{author}/{category}
MyBookSearchMS	http://localhost:8000/mybook/{bookId}
MyBookSearchMS	http://localhost:8000/updateBookRating
MyBookSearchMS	http://localhost:8000/updateBookInventory